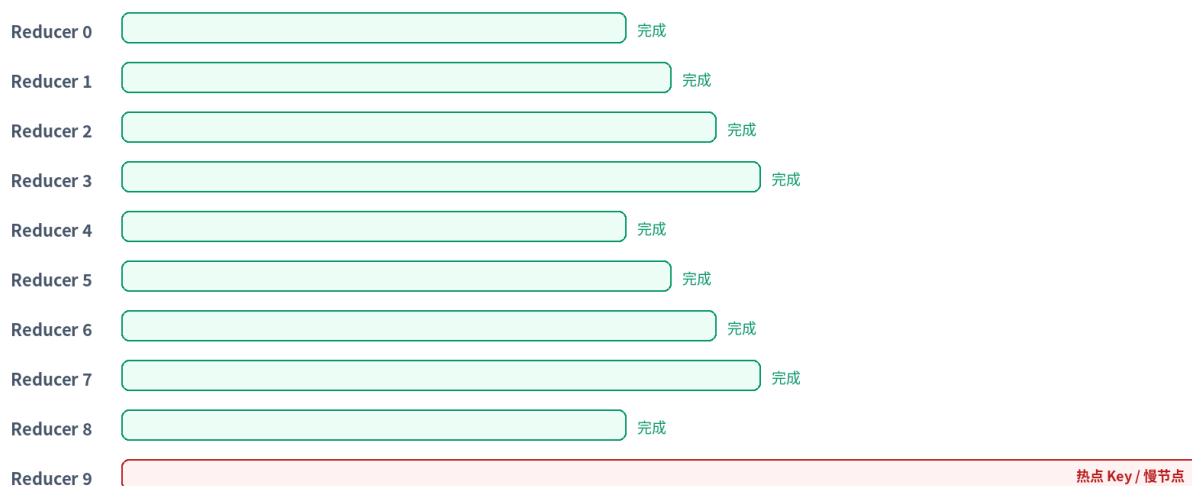


Hadoop 中的 MapReduce 原理详解

第六讲：可靠性、长尾任务与性能优化

Task Retry · Speculative Execution · Data Skew · Combiner · Compression
· Parallelism

为什么 Job 会“99% 卡很久”：整体完成时间由最慢任务决定



底层机制：并行任务的 wall-clock time 由“长尾任务（straggler）”控制；进度条接近 100% 不代表剩余工作量很少
9 个 Reducer 已完成 ≠ Job 完成
最后一个 Reducer 决定总时长

目标：理解为什么“能跑完”和“跑得快”是两个问题，以及真实 Job 为什么常常卡在最后一点。

学习目标

前五讲已经把 MapReduce 的数据流和 YARN 控制流串起来。本讲开始进入“真实集群为什么有时很慢、为什么机器坏了还能继续、为什么进度 99% 可能还要等很久”的问题。这些现象本质上都来自分布式系统的长尾、故障和数据不均衡。

- 区分 Task 失败重试与推测执行：前者处理失败，后者处理异常慢。
- 理解“长尾任务（straggler）”为什么决定整个 Job 的 wall-clock time。
- 理解数据倾斜如何从业务 Key 分布一路传导成某个 Reducer 特别慢。
- 理解 Combiner 的价值与限制，以及为什么它不能承担程序正确性。
- 理解 Map 输出压缩为什么可以用 CPU 换网络/磁盘 I/O。
- 理解 Mapper/Reducer 并行度不是越大越好，而是吞吐、调度开销和资源竞争之间的平衡。

- 掌握一个真实排查框架：看到 99% 卡住时，先判断“失败、慢节点、数据倾斜、Shuffle、GC/内存、资源不足”中的哪一类。

先记住一个总原则

分布式计算的总耗时，不由“平均任务”决定，而经常由“最慢的那几个任务”决定。优化 MapReduce，核心就是：减少失败、减少数据搬运、让负载更均匀、避免长尾。

1. 为什么 MapReduce 能容错：失败了就重算

MapReduce 的设计假设不是“机器不会坏”，而是“机器迟早会坏”。当数据规模扩展到数百、数千台普通服务器以后，磁盘、网络、进程、节点发生故障并不罕见。因此框架把“失败重试”作为常态机制。

1.1 Task 与 TaskAttempt 再复习一次

Task 是逻辑工作，例如 MapTask-3。TaskAttempt 是一次实际执行，例如 attempt 0 在 Node A 上运行。如果 attempt 0 失败，MRAppMaster 可以创建 attempt 1，让同一个逻辑 Task 在另一个 Container 里重新做。

Hadoop 3.3.5 的 JobConf API 中，Map task 和 Reduce task 的最大尝试次数在未另行配置时都默认为 4。理解这个数字的重点不是背配置，而是认识到：单次 Attempt 失败，并不等于整个 Job 立即失败。

常见相关配置项：

```
mapreduce.map.maxattempts  
mapreduce.reduce.maxattempts
```

1.2 为什么 Map 特别适合重算？

因为 Map 的原始输入通常来自 HDFS。HDFS 保存了数据副本，而 Map 函数通常被设计成确定性的局部计算。即使某个节点上的 Map 中间结果丢失，也可以重新从 HDFS 读取相同 Split，再执行一次 Mapper。

本质

MapReduce 选择的是“原始数据可靠保存 + 临时计算结果可以重做”，而不是“每一份 Map 中间结果都做昂贵多副本”。这是用计算成本换取存储和复制成本。

2. 推测执行：任务没失败，但为什么还要再跑一份？

重试 vs 推测执行：一个处理“失败”，一个处理“异常慢”



图 1：Task Retry 与 Speculative Execution 解决的是两类不同问题。

现实中最麻烦的不只是“死掉的任务”，还有“活着但特别慢的任务”。比如 100 个 MapTask 中 99 个都在 2 分钟内完成，最后一个因为磁盘故障预兆、CPU 抢占、GC、网络慢等原因跑了 20 分钟。如果什么都不做，整个 Job 必须等它。

2.1 推测执行的基本思想

框架观察到某个 Task 的进度明显落后于同类任务时，可以在另一台机器上为同一个逻辑 Task 再启动一个 Attempt。两个 Attempt 同时竞争，谁先成功，框架就采用谁的结果，并终止多余的那个。

Hadoop 3.3.5 API 提供 map/reduce 两侧的 speculative execution 开关；JobConf API 文档中 map 与 reduce 的推测执行默认值为 true。不过真实生产集群可能在站点配置或 Job 级别关闭它，因此不要把“默认值”误解成“任何集群一定开启”。

相关配置：

```
mapreduce.map.speculative
mapreduce.reduce.speculative
```

2.2 为什么推测执行不是免费加速？

因为它会复制计算。原任务其实可能只是暂时慢，新 Attempt 又占用了一个 Container、CPU、磁盘和网络。资源紧张时，过多推测任务反而可能让其他正常任务更慢。

特别注意 Reduce

Reduce 的后期可能已经做了大量 Shuffle 和计算，复制一个 Reduce Attempt 的代价比复制短小 Map 大得多。因此推测执行是一种“对付长尾的保险机制”，不是无脑开启越多越好。

3. 为什么会出现“99% 卡很久”？

为什么 Job 会“99% 卡很久”：整体完成时间由最慢任务决定

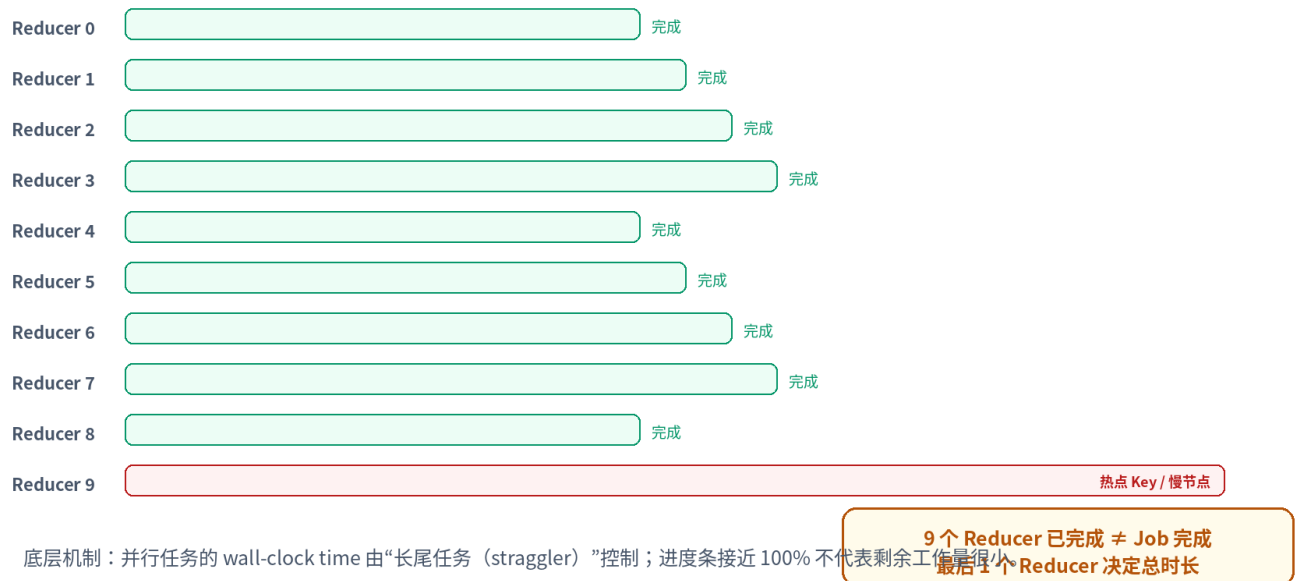


图 2：并行系统的总完成时间由最慢任务决定。

进度条容易给人一种错觉：99% 意味着只剩 1% 的时间。实际上 MapReduce 的进度不是简单线性倒计时。假设 10 个 Reducer 里 9 个已经结束，最后一个 Reducer 吃到了远多于其他 Reducer 的数据，它可能还需要很久。

3.1 常见原因

现象来源	底层原因	你可能看到的表现
数据倾斜	某个 partition / key 数据量巨大	一个 Reducer 长时间 100% 忙，其他 Reducer 早已结束
慢节点	磁盘、CPU、网络或宿主机异常	某个 TaskAttempt 比同类明显慢
Shuffle 慢	Map 输出大、网络拥塞、Fetch 重试	Reducer 长时间停留在 shuffle/sort 阶段
内存/GC	对象太多、堆不足、频繁溢写/GC	CPU 高但有效进度慢，日志出现 GC 或 OOM
资源不足	Container 排队、集群竞争	任务并未真正运行，而是在等待资源
代码逻辑	单个 key 的 reduce() 本身很慢	某个 key 长时间处理不完

所以看到“99%”时不要只盯百分比。真正要问的是：剩下的是哪个 Task？它在哪个阶段？输入量多少？是不是某个 Key 特别大？有没有失败重试或 Fetch Failure？

4. 数据倾斜：MapReduce 最典型的性能杀手之一

数据倾斜：Hash 分区“公式正确”，也可能业务上极不均匀

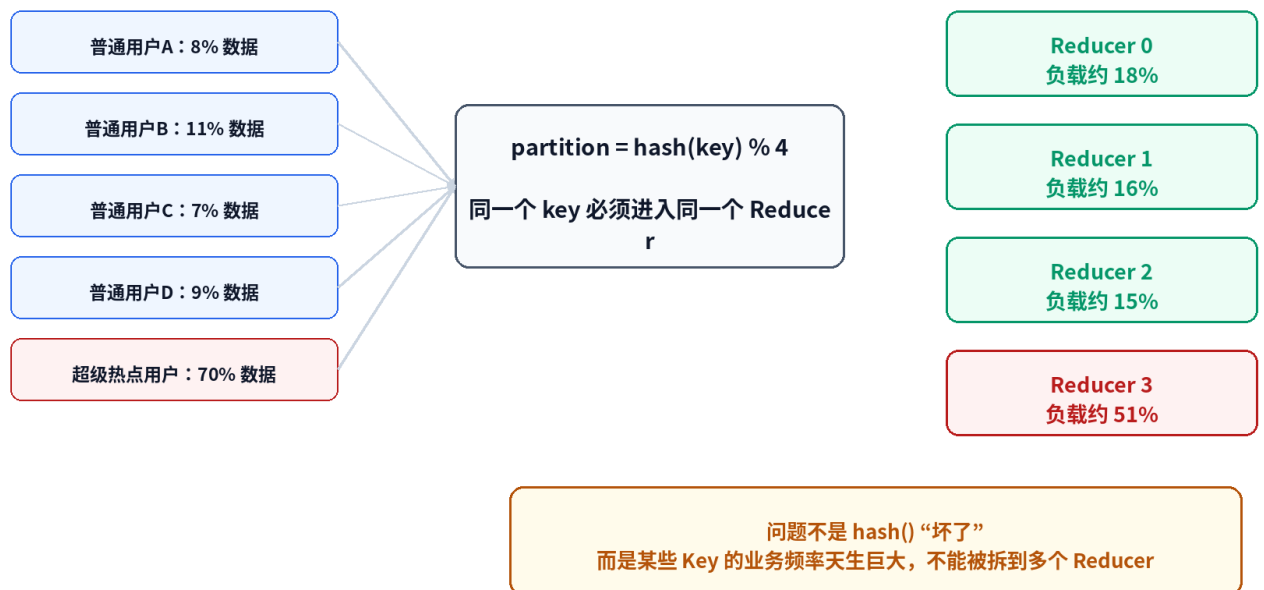


图 3：HashPartitioner 只能保证“同 Key 到同 Reducer”，不能保证业务数据天然均匀。

4.1 为什么 Hash 分区也会倾斜？

默认 HashPartitioner 的逻辑可以粗略理解为 $\text{hash}(\text{key})$ 再取模。对于大量、频率相近的 Key，这通常能把 Key 分散到不同 Reducer。但是同一个 Key 必须进入同一个 Reducer。假设“超级热点用户”占全部记录的 40%，它无论 hash 到哪个 partition，那个 Reducer 都会非常重。

4.2 一个现实例子：统计商品销量

普通商品每天几千条订单，而某次直播爆款有 5,000 万条订单。Mapper 输出 $\langle \text{商品 ID}, 1 \rangle$ 。爆款商品的全部 5,000 万个 value 必须聚到同一个 Reducer。其他 Reducer 可能几分钟完成，这个 Reducer 却要处理很久。

4.3 常见解决思路

- 热点 Key 加随机前缀：先把一个热点 Key 拆成多个子 Key，第一阶段局部聚合，第二阶段再合并。
- 自定义 Partitioner：当业务 Key 结构已知时，让分区策略更符合数据分布。
- Map 端预聚合 / Combiner：如果操作满足条件，先把大量重复 Key 在 Mapper 本地合并。
- 采样后再分区：先估计数据分布，再设计边界或分区规则。
- 改变问题设计：某些“必须所有数据聚到同一个 key”的计算天然不适合直接一阶段 Reduce。

重要区别

“增加 Reducer 数量”不一定能解决单个热点 Key。因为同一个 Key 仍然只能进入一个 Reducer。热点 Key 不拆，Reducer 从 10 个加到 100 个也可能仍然有一个特别慢。

5. Combiner：最便宜的优化之一，但有严格前提

Combiner 的目的就是在数据离开 Mapper 之前先做局部聚合，从而减少 Spill 文件和 Shuffle 网络流量。第三讲已经见过它，这里从性能角度再看一次。

WordCount 的理想情况：

Mapper 原始输出：

```
apple -> 1
apple -> 1
apple -> 1
banana -> 1
```

Combiner 后：

```
apple -> 3
banana -> 1
```

假设 1000 万条记录中大量 Key 重复，Combiner 可以把“千万级 pair”减少成“每个 Mapper 每个 Key 一条局部结果”。网络传输量可能大幅下降。

5.1 为什么 Combiner 不能保证执行？

框架可以选择执行 0 次、1 次或多次 Combiner；你的业务结果不能依赖“它一定运行”。因此 Combiner 必须满足：无论做不做、做几次，最终 Reduce 结果都应该相同。

5.2 SUM 为什么适合，直接 AVERAGE 为什么危险？

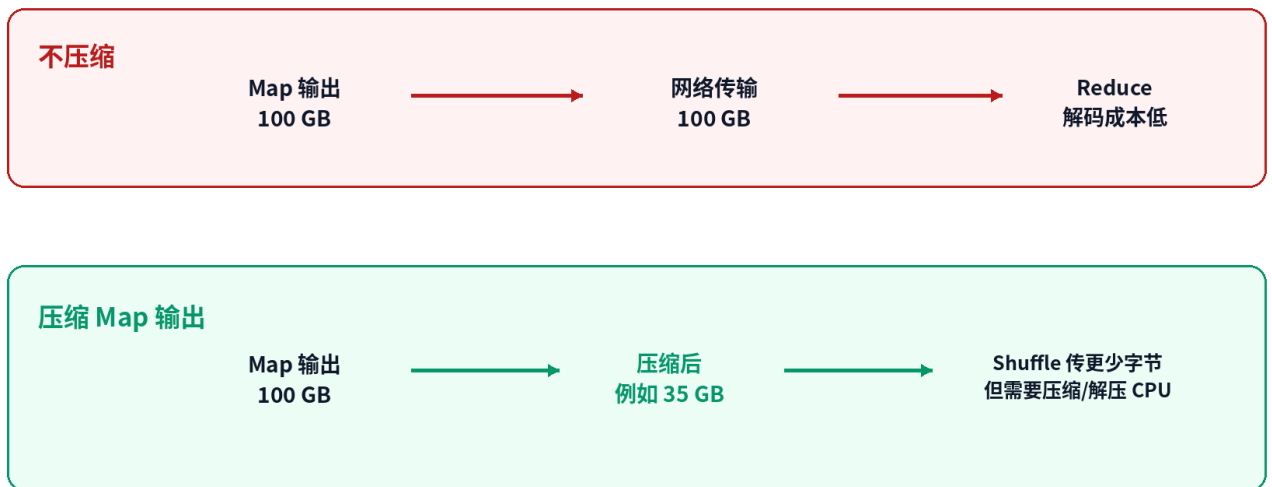
求和具有很好的可组合性：局部和再求和仍是总和。平均值如果只传一个局部平均值，就会丢失样本数量。正确做法通常是传 <sum,count>，最后用总 sum / 总 count。

判断口诀

能不能做 Combiner，不要问“Reducer 代码能不能复制过去”，而要问“局部聚合的结果能不能继续被同一种规则安全地再次聚合”。

6. 压缩：为什么 Map 输出压缩经常很值？

压缩为什么常常能加速 Shuffle：用 CPU 换网络 and 磁盘 I/O



是否更快取决于瓶颈：网络/磁盘紧张时通常划算；CPU 已经满载、数据本身难压缩时收益可能较小。

图 4：压缩的本质是 CPU 与网络/磁盘 I/O 的交换。

Shuffle 往往意味着大量中间结果通过网络搬运。如果数据可压缩，那么 Mapper 本地先压缩 Map 输出，Reducer Fetch 的就是更少字节。网络 and 磁盘通常比 CPU 更容易成为大数据批处理瓶颈，因此这种交换经常划算。

常见配置项：

```
mapreduce.map.output.compress
mapreduce.map.output.compress.codec

mapreduce.output.fileoutputformat.compress
mapreduce.output.fileoutputformat.compress.codec
```

Hadoop 3.3.5 API 明确提供 Map 输出压缩和最终 FileOutputFormat 输出压缩的配置。前者主要影响 Shuffle，中间结果不会成为最终 HDFS 文件；后者影响最终结果文件的存储体积和后续读取。

6.1 哪些情况下收益大？

- 文本、JSON、日志等重复性强、压缩比高的数据。
- 网络较慢或跨机架 Shuffle 很重。
- 磁盘 I/O 已成为瓶颈。
- CPU 还有余量。

6.2 哪些情况下可能不划算？

- 数据本身已经压缩（例如某些压缩媒体文件）。
- CPU 已经满载，压缩算法又很重。
- 中间结果很小，压缩节省的字节抵不过编码/解码开销。

7. 并行度：Mapper / Reducer 是不是越多越快？

不是。并行度太小，机器吃不满；并行度太大，任务启动、调度、JVM/Container、文件、Shuffle 连接等管理成本又会上升。真正目标是让“每个任务工作量足够大，同时有足够多任务把集群资源利用起来”。

7.1 Mapper 数主要受什么影响？

对于 FileInputFormat，Mapper 数主要来自 InputSplit 数量。Split 又受到文件大小、HDFS block、min/max split size、输入格式是否可切分等因素影响。不能简单依靠“我想设 100 个 Mapper”来精确控制。

7.2 Reducer 数为什么更像一个调参旋钮？

Reducer 数通常由 Job 配置明确决定。太少：每个 Reducer 数据太多、并行度不足；太多：产生更多任务启动开销、Shuffle 连接和输出 part 文件，甚至形成“小文件问题”。

Reducer 太少	Reducer 太多
单个 partition 很大	任务启动和调度开销增加
某个 Reducer 容易成为瓶颈	产生大量 part-r 文件
CPU/网络并行度不足	每个任务数据太少，效率下降
失败重算代价大	Shuffle 连接和元数据开销增加

不要机械套公式

官方 JobConf 文档给过 reducer 数量的经验性建议，但真实最优值依赖集群资源、数据量、Key 分布、Shuffle 体积和 SLA。教学阶段应该先理解“每个 Reducer 的目标工作量”，而不是背某个固定倍数。

8. 内存、Spill、Merge：为什么“内存加大”有时能快很多？

第三、四讲讲过 Map 输出缓冲、Spill 和 Reduce 端 merge。它们的共同规律是：内存装不下就落盘，落盘文件太多又要多轮 merge。磁盘 I/O 和 merge 次数越多，耗时越大。

常见性能相关参数（不要在不了解集群时盲目修改）：

```
mapreduce.task.io.sort.mb
mapreduce.task.io.sort.factor
mapreduce.reduce.shuffle.parallelcopies
```

Apache Hadoop 3.3.5 的集群配置示例也把 io.sort.mb、io.sort.factor 和 reduce.shuffle.parallelcopies 作为大型集群可调整的 MapReduce 参数。但“调大”会同时增加内存或连接压力，因此必须结合 Container 资源和数据规模。

8.1 内存优化的真正目标

- 减少 Map 端频繁 Spill。

- 减少多个 Spill 文件的多轮 Merge。
- 让 Reduce Shuffle 更多数据在内存中完成合并。
- 避免堆太小导致频繁 GC 或 OOM。

8.2 为什么内存也不是越大越好？

单个 Container 占用更大内存后，同一节点能并行运行的 Container 数量会下降。你可能让一个 Task 快了，却让整个集群并行度下降。因此性能优化永远是“单任务效率”和“集群吞吐”的平衡。

9. 四个性能杠杆：看到慢 Job 时先从这里想

MapReduce 性能优化的四个基本杠杆



图 5：绝大多数 MapReduce 优化最终都可归入四类：少产生、少搬、均衡、合适并行。

9.1 少产生数据

越早过滤越好。一个 Mapper 能在本地丢掉无用记录，就不要把无用 pair 送进 Shuffle。可以局部聚合时，尽量减少中间 pair 数量。

9.2 少搬字节

Shuffle 是昂贵网络 I/O。Map 输出压缩、合理序列化、Combiner、只输出必要字段，本质上都在减少“网络上的字节数”。

9.3 分得更均匀

平均并不等于没有长尾。真正要观察每个 partition 的记录数/字节数和热点 Key。均匀负载往往比单纯增加机器更有效。

9.4 并行度合适

让任务数量足以利用机器，但不要碎成海量小任务。一个好的分布式作业应该让每个 Task 都有“值得启动一次 Container”的工作量。

10. 看到“99% 卡住”时，怎么一步一步排查？

把排查过程固定成一个闭环，而不是凭感觉调参数。下面这个顺序非常适合课堂和以后真实使用。

步骤	先看什么	想回答的问题
1	YARN / Job 状态	到底是 Map 慢还是 Reduce 慢？有没有 Task 重试？
2	各 Task duration	是不是只有 1-2 个 Task 比其他任务慢很多？
3	Task counters / input bytes	慢 Task 是不是吃了异常多数据？
4	Shuffle counters	Map output / Reduce shuffle bytes 是否特别大？
5	日志	有没有 Fetch Failure、OOM、GC、磁盘/网络错误？
6	Key 分布	是否存在热点 Key / partition 倾斜？
7	资源	Container 是否排队？节点 CPU/内存/磁盘是否饱和？
8	再决定优化	Combiner、压缩、分区、并行度、内存，哪一个真正对应根因？

反例

“任务慢 -> 把内存调大”不是诊断。数据倾斜时，内存调大可能一点用都没有；如果慢是因为网络 Shuffle，真正有效的可能是 Combiner、压缩或减少中间输出。

11. 在你的 Hadoop 3.3.5 虚拟机里做一个观察实验

单机伪分布式环境不容易制造真实多节点慢机，但仍可以观察 Job counters、任务数量、输出文件和配置值。每次重开虚拟机后先启动 HDFS/YARN 并确认健康。

A. 启动服务并检查

```
start-dfs.sh
start-yarn.sh
jps

yarn node -list
```

B. 看看你自己的环境有没有覆盖默认配置

```
# 查看 Hadoop 当前配置中某个参数的解析值（如果命令/配置环境支持）
hadoop getconf -confKey mapreduce.framework.name

# 也可以直接查看本机配置文件
grep -n "mapreduce.*speculative\|mapreduce.*maxattempts\|mapreduce.*compress" \
$HADOOP_HOME/etc/hadoop/mapred-site.xml
```

注意：mapred-site.xml 里“没有写”不等于功能不存在，可能表示使用 Hadoop 默认值。教学时要让学生区分“站点显式配置”和“框架默认配置”。

C. 观察 counters

```
# 跑 WordCount 后，可从控制台输出或 YARN/History 页面观察 counters
# 重点关注的关键词包括：
# Map input records
# Map output records
# Map output bytes
# Reduce shuffle bytes
# Spilled Records
# Failed Shuffles / Merge 等

hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.5.jar \
wordcount /mrdemo/input /mrdemo/output6
```

最有价值的不是“跑成功”，而是比较 Map input records 与 Map output records、Map output bytes 与 Reduce shuffle bytes，再回忆 Combiner、压缩和 Spill 为什么会改变这些数字。

12. 常见疑问与容易理解错的地方

Q1: Task 失败重试和推测执行是不是一回事？

不是。重试通常发生在原 Attempt 已失败后；推测执行是原 Attempt 还在跑，只是被判断为异常慢，于是并行启动另一个 Attempt。

Q2: 推测执行会让同一个 Task 同时跑两份吗？

可能会。最终只接受一个成功结果，多余 Attempt 会被终止。

Q3: 为什么不把所有任务都复制两份，肯定更快？

资源成本会翻倍，并产生更强的 CPU、磁盘和网络竞争。推测执行只针对长尾候选，而不是全量复制。

Q4: 99% 卡住是不是 Hadoop 出 bug？

不一定。最常见原因之一是长尾 Reducer、数据倾斜或 Shuffle/资源问题。

Q5: Reducer 越多越能解决倾斜吗？

不一定。单个热点 Key 无法靠增加 Reducer 数自动拆开。

Q6: Combiner 能保证减少网络吗？

不能保证一定执行；而且减少程度取决于每个 Mapper 内部 Key 的重复程度。

Q7: 为什么平均值不能直接用平均数做 Combiner？

局部平均的平均通常不等于全局平均；必须保留权重/计数，例如 <sum,count>。

Q8: 压缩是不是肯定加速？

不是。它节省 I/O，但消耗 CPU；瓶颈不同，收益不同。

Q9: 最终输出压缩和 Map 输出压缩有什么区别？

Map 输出压缩主要减少 Shuffle；最终输出压缩主要减少 HDFS 存储和后续读取字节。

Q10: 内存越大 Job 越快吗？

不一定。单 Task 更大内存可能减少 Spill，但会降低每台机器可并发 Container 数。

Q11: Mapper 越多越好吗？

不是。小任务过多会增加调度、启动、文件与管理开销。

Q12: Reducer 数为 1 为什么有时特别慢？

所有中间数据最终都进入一个 Reducer，完全失去 Reduce 阶段的并行性，但有时业务确实需要单点全局结果。

Q13: 数据倾斜和节点慢怎么区分？

倾斜通常表现为慢 Task 的输入/Shuffle 数据量明显更大；慢节点则可能工作量相近但运行速度异常慢。

Q14: 优化最重要的是调参数吗？

通常不是。数据结构、Key 设计、是否产生大量无用中间数据，比微调参数更具决定性。

13. 本讲最终心智模型

MapReduce 的可靠性来自“失败可检测、Task 可重试、原始输入可重读”；性能则主要受“数据量、网络搬运、负载均衡、并行度和长尾”支配。两者结合起来，你就能解释绝大多数真实 Job 的异常现象。

可靠性：

失败 Attempt -> 重新调度 -> 新 Attempt -> 完成逻辑 Task

长尾控制：

异常慢 Attempt -> 可能推测执行 -> 两个 Attempt 竞速 -> 接受先成功者

性能主线：

少产生数据 + 少搬字节 + 分区均衡 + 并行度合适

一句话总结

MapReduce 优化的本质不是“把某个参数调大”，而是让更多机器做真正有用且相对均匀的工作，同时尽量减少磁盘和网络上的无效数据搬运。

下一讲建议进入 MapReduce 编程模型：Driver、Mapper、Reducer、Writable、Context，以及一份 Java WordCount 代码中每一行到底对应前六讲的哪个底层机制。这样就从“懂原理”正式进入“看得懂并能写程序”。

资料依据

1. Apache Hadoop 3.3.5 MapReduce / JobConf API：Task attempt、speculative execution、Reducer 数量等接口与默认行为说明。
2. Apache Hadoop 3.3.5 FileOutputFormat API：最终输出压缩配置。
3. Apache Hadoop 3.3.5 Cluster Setup：MapReduce 大型集群配置示例，包括 io.sort.mb、io.sort.factor、reduce.shuffle.parallelcopies 等。

说明：本文讲的是 Hadoop 3.x / YARN MapReduce 的核心机制。生产集群的 speculative、重试次数、资源额度、压缩 codec 等可能被管理员覆盖，应以实际 mapred-site.xml 和 Job 配置为准。